SkyVoyage / Flight Booking Website — Explanation (Part 1/5)

Part 1: Project overview, folder structure, and database schema(s).

1) What this project is

This repository is a flight booking platform with: (1) a PHP backend (REST-style endpoints under `backend/api`), (2) a frontend made of static HTML/CSS/JS pages under `frontend`, and (3) SQL scripts under `database`.

Users can register/login as either a **company** (adds flights, views passengers, cancels flights/refunds) or a **passenger** (searches flights, books, messages companies).

2) Folder structure (what each folder does)

Path	Purpose
`backend/config`	Backend configuration: database connection, app settings, CORS headers.
`backend/models`	PHP “Model” classes for data logic (note: these models match an older/alternate schema).
`backend/api`	HTTP endpoints used by the frontend (JSON responses).
`frontend`	Static pages (HTML) + styling (CSS) + browser logic (JS).
`database`	SQL scripts to create/seed the database schema.
`Flight-Booking-Website/`	**Empty folder** in this workspace (safe to ignore; not used by code).

3) How the app is supposed to run (high level)

1. Import a SQL schema into MySQL/MariaDB.
2. Configure DB credentials in `backend/config/database.php`.
3. Serve the project via Apache (XAMPP/WAMP) so the browser can load `frontend/*.html` and call `backend/api/*.php`.
4. Frontend pages call backend endpoints using `fetch()` and store session info in `localStorage`.

4) Important: there are TWO different database “worlds” in this repo

The codebase contains two sets of assumptions about the database tables. You must understand this to avoid confusion during testing.

- **World A (Project Requirements / Company–Passenger):** schema in `database/project_schema.sql`. Tables: `users` (company/passenger), `flights` (company_id, fees, passenger counts), `flight_itinerary` (cities and order), `bookings`, `messages`.
- **World B (Airports/Airlines/Seat-class):** implied by `backend/models/Flight.php`, `backend/models/Booking.php`, `backend/models/User.php` and by some endpoints like `backend/api/airports.php` and `backend/api/flights.php?action=search`. This world expects tables like `airports`, `airlines`, seat columns like `available_seats_economy`, etc.

In practice, the “dashboard” pages (company/passenger) and endpoints such as `register.php`, `auth.php`, `search-flights.php`, `company-flights.php`, `company-flight-details.php`, `passenger-bookings.php`, `messages.php` are aligned with **World A**.

Some other pages (like `search-results.html`) and the generic API module (`frontend/js/api.js`) are aligned with **World B**.

5) Database: World A (database/project_schema.sql)

This is the schema used by the “project requirements” endpoints.

5.1 users

- `user_type`: enum('company','passenger')
- `name,email,password,tel`: shared identity fields
- Company fields: `bio,address,location,logo_img`
- Passenger fields: `photo,passport_img`
- `account_balance`: wallet-like balance used by bookings/refunds

5.2 flights

- Created by companies: `company_id` foreign key to users
- `fees`: ticket price per passenger
- `max_passengers`, `registered_passengers`, `pending_passengers`
- `is_completed`: used as completed/cancelled flag in UI

5.3 flight_itinerary

- Stores route as ordered cities: `city` + `sequence_order`.
- Also includes `start_datetime` and `end_datetime` in the schema file.
- Some PHP endpoints insert only (flight_id, city, sequence_order); date-time fields may need defaults or schema adjustment if strict.

5.4 bookings

- `status`: pending / registered / cancelled
- `payment_type`: account / cash
- `amount_paid`: the fee paid

5.5 messages

- Messages between users with optional `flight_id` context.
- `is_read` exists in schema; current endpoints do not update it.

6) Database: World B (database/flight_booking_export.sql)

This file is a database dump (partial in this repo view) that includes objects like a view `active_bookings_summary`. The models under `backend/models` strongly suggest the presence of tables such as `airports`, `airlines`, `aircraft`, seat-availability columns, etc.

If you want to run the “airports + seat classes” search flow, you likely need to import the export file and confirm that all those tables exist.

7) Files covered in Part 1 (nothing skipped)

- `README.md` — high-level features, setup notes.
- `database/project_schema.sql` — World A schema + sample seed data.
- `database/flight_booking_export.sql` — World B dump (airports/airlines ecosystem).
- `database/project_schema.sql` and `database/flight_booking_export.sql` exist; `database/flight_booking.sql` does not exist in this workspace even though README mentions it.
- `Flight-Booking-Website/` — empty folder (unused).

End of Part 1/5.